

```
1 // INFIX TO POSTFIX CONVERSION
2
3 #include <stdio.h>
4 #include <ctype.h>
5 #include <string.h>
6 #define MAX 100
7 char stack[MAX];
8 int top = -1;
9
10 // Push function
11 void push(char c) {
12     if (top == MAX - 1) {
13         printf("Stack Overflow\n");
14         return;
15     }
16     stack[++top] = c;
17 }
18
19 // Pop function
20 char pop() {
21     if (top == -1) {
22         printf("Stack Underflow\n");
23         return -1;
24     }
25     return stack[top--];
26 }
27
28 //peek function
29 char peek() {
30     if (top == -1) return -1;
31     return stack[top];
32 }
33
34 // Function to return precedence of operators
35 int precedence(char op) {
36     switch (op) {
37         case '+':
38         case '-':
```

```
main(): int
Start here x LABPGM2.c x
37         case '-':
38             return 1;
39         case '*':
40             case '/':
41                 return 2;
42         case '^':
43             return 3; // Highest precedence
44         case '(':
45             return 0;
46     }
47     return -1;
48 }
49
50 // Function to return associativity
51 // 0 = Left-to-Right, 1 = Right-to-Left
52 int associativity(char op) {
53     if (op == '^')
54         return 1; // Right-to-Left
55     return 0; // +, -, *, / - Left-to-Right
56 }
57 // Function to convert infix to postfix
58 void infixToPostfix(char infix[], char postfix[]) {
59     int i, k = 0;
60     char c;
61
62     for (i = 0; infix[i] != '\0'; i++) {
63         c = infix[i];
64
65         if (isalnum(c)) {
66             // Operand - directly to postfix
67             postfix[k++] = c;
68         }
69         else if (c == '(') {
70             push(c);
71         }
72         else if (c == ')') {
73             while (peek() != '(') {
74                 postfix[k++] = pop();
75             }
76             pop(); // discard '('
```

```

75         }
76         pop(); // discard '('
77     }
78     else {
79         // Operator
80         while (top != -1 &&
81             ((precedence(peek()) > precedence(c)) ||
82             (precedence(peek()) == precedence(c) &&
83             associativity(c) == 0))) { // L-to-R
84             postfix[k++] = pop();
85         }
86         push(c);
87     }
88 }
89
90 // Pop remaining operators
91 while (top != -1) {
92     postfix[k++] = pop();
93 }
94 postfix[k] = '\0';
95 }
96
97 int main() {
98     char infix[MAX], postfix[MAX];
99     printf("Enter a valid parenthesized infix expression: ");
100     scanf("%s", infix);
101     infixToPostfix(infix, postfix);
102     printf("Postfix Expression: %s\n", postfix);
103     return 0;
104 }
105

```

```

C:\Users\student\Desktop\1BI X + v
Enter a valid parenthesized infix expression: (A+(B*C-(D/E^F)*G)*H)
Postfix Expression: ABC*DEF^/G*-H*+

Process returned 0 (0x0)   execution time : 14.516 s
Press any key to continue.

```



C:\Users\student\Desktop\1Bl



Enter a valid parenthesized infix expression: A*B+C*D-E
Postfix Expression: AB*CD*+E-

Process returned 0 (0x0) execution time : 13.237 s
Press any key to continue.